ARCHITECTURE: FROM DATA SOURCE TO SCREEN

Aurora: climate and AI forecast platform

Eighty-six years of daily weather for 40 cities flow through a medallion lakehouse on Databricks into a tested star schema, a seven-day machine-learning forecast with a stated range, and a public dashboard.

✓ Built, tested and deployed. Runs unattended every day at 05:30 UTC.

Cities, on six continents
40

Years of daily records
86

Data tests on every run
23

Days forecast, with a range
7

1 **Sources** Open-Meteo APIs › 2 **Ingest** Python, Lakeflow Jobs › 3 **Medallion** Lakeflow pipeline, dbt › 4 **Forecast** LightGBM, MLflow › 5 **Serve** Parquet, FastAPI, DuckDB › 6 **Show** React dashboard

7 **DataOps and infrastructure** underpin every stage: continuous integration and deployment, infrastructure as code, two environments, monitoring and secrets

1 SOURCES

Free, open, credited data

Open-Meteo Historical Weather API

DONE

ERA5 reanalysis · no API key · CC BY 4.0

- Five daily values per city: mean, highest and lowest temperature, total precipitation and strongest wind gust
- From 1 January 1940 to about five days ago: roughly 31,700 days per city
- Free tier of about 10,000 weighted calls a day; one city's full history costs about 1,130

Open-Meteo Geocoding API

DONE

City name to coordinates · no API key

- Resolves each city to latitude, longitude, elevation and a stable location ID
- Takes the most populous match within the given country code
- All 40 production cities were checked against it before going live

City lists as configuration

DONE

CSV files in the repository

- `config/locations.csv`: 7 cities for development
- `config/locations_prod.csv`: 40 cities on six continents for production
- Each environment is handed its own list by the deployment bundle

2 INGESTION AND ORCHESTRATION

Self-managing, scheduled, serverless

Self-managing extraction

DONE

Python · Databricks serverless compute

- `extract_weather.py` reads the silver table to see which cities already have history
- Loaded cities get a cheap 14-day refresh first; then up to four new cities are backfilled per run
- A per-minute refusal waits 60 seconds and retries; an hourly or daily one stops cleanly and logs the API's reason

Raw landing zone

DONE

Unity Catalog volume · JSON lines

- API replies are written unchanged to `/Volumes/<catalog>/bronze/landing`
- Every file is stamped with its extraction time, which later decides which version of a day wins
- Any later stage can be rebuilt from here without calling the API again

One daily job

DONE

Lakeflow Jobs · every day at 05:30 UTC

- `aurora_daily` runs five stages in order; each is also a job of its own, so any one can be re-run alone
- A failure stops the chain, the site keeps yesterday's data and an email goes out
- Nothing runs on a laptop

TASK CHAIN `ingest` locations, then weather › `bronze_silver` Lakeflow pipeline › `gold` dbt build and tests › `score` seven-day forecast › `serving` export and delivery

3 MEDALLION LAKEHOUSE

Raw, cleaned, modelled: bronze, silver, gold on Delta Lake

Bronze: raw

DONE

Lakeflow Declarative Pipeline · Auto Loader

- Auto Loader picks up only the files that are new since the last run
- Each reply is kept whole in one VARIANT column, so nothing from the source is lost
- Streaming tables: `openmeteo_daily_raw`, `locations_raw`

›

Silver: cleaned

DONE

Lakeflow AUTO CDC · pipeline expectations

- Daily arrays are unpacked to one row per city per day
- AUTO CDC keyed on city and date, sequenced by extraction time: re-runs and corrections replace rows and never duplicate them
- Expectations drop any row with a missing key or an incomplete day
- Tables: `weather_daily`, `locations`

›

Gold: modelled

DONE

dbt · SQL warehouse · star schema

- `dim_location`, `dim_date` and `fact_weather_daily`, built incrementally with MERGE and liquid clustering
- `climate_baseline`: the 1961 to 1990 mean, spread and 90th percentile for every city and day of the year, smoothed over 15 days
- `fact_climate_anomaly`: degrees from normal, hot days, and warm spells of three or more days in a row
- 23 dbt tests on every run: unique, not null, relationships and a duplicate-fact check

Real issue found and fixed. The archive's first day, 1 January 1940, arrives incomplete for cities east of UTC. An expectation now drops those rows at the door instead of letting nulls into the cleaned table.

4 MACHINE LEARNING

A seven-day temperature forecast with a stated range, tested before it was trusted

Features

DONE

dbt · ml.features_temperature

- One row per city per day, using only what was known that day
- Anomaly today and 1, 2, 3 and 7 days back, 7- and 30-day means, 7-day spread
- Rain over 7 days, wind gust, daily range, time of year, latitude and elevation
- Targets: the anomaly 1 to 7 days ahead

Backtest

DONE

LightGBM · MLflow tracking

- Tested on 2023, 2024 and 2025, each time trained only on data from before the test dates
- Day 1 typical miss: 1.32 °C, against 1.47 °C for assuming no change and 2.04 °C for the recent normal
- Day 7: 2.10 °C, against 2.73 °C and 2.16 °C. The edge has nearly gone
- The 80% ranges held the truth 77 to 80% of the time

Train and register

DONE

Quantile regression · Unity Catalog registry

- Three gradient-boosted models for the 10th, 50th and 90th percentiles, trained on data from 1980 onwards
- Wrapped as one MLflow model, so a single call returns low, most likely and high
- Registered as `ml.temperature_forecaster`; the alias `champion` marks the version in use

Daily scoring

DONE

Batch job · gold.fact_forecast

- Loads the champion and the newest feature row for each city
- Predicts seven horizons, then adds the normal back to give degrees Celsius
- Written with MERGE, so a re-run never duplicates a forecast
- Retraining stays manual until all 40 cities are loaded and a quality gate exists

Real issue found and fixed. The first model registration failed with an S3 AccessDenied on Free Edition's default storage. Routing artifact uploads through the Databricks SDK, with one environment variable set before MLflow is imported, fixed it.

5 SERVING AND DELIVERY

Small files, a small API and no database server to run

Serving tables

DONE

dbt · serving schema

- Five small tables shaped for exactly what the page shows
- `city_summary`, `city_yearly`, `city_recent_daily`, `forecast_latest`, `model_skill`
- Cities still waiting for history appear flagged, not hidden

Export

DONE

PySpark · Parquet · Unity Catalog volume

- Each table is written as one Parquet file: about 80 KB in all for 40 cities
- A manifest is written last with every row count, so its presence means the export is complete

Push delivery

DONE

HTTPS · token from a secret scope

- The job uploads each file to the site, which holds it in a staging folder
- It then sends the manifest. The site counts each file's rows against it and only then swaps the files in
- The web server holds no Databricks credentials at all
- Development has no upload address, so it cannot reach the live site

API

DONE

FastAPI · DuckDB · Python 3.12

- DuckDB reads the Parquet files in-process: there is no database server to run
- Six read endpoints: health, cities, a city's yearly history, its recent days, its forecast, and model skill
- Parameterised queries throughout; uploads capped at 5 MB and limited to known table names

Real issue found and fixed. Cloudflare was replacing the API's error replies with a web page marked as a success. The export job and the dashboard now both check that a reply is real API output before trusting it.

6 CONSUMPTION: THE PUBLIC DASHBOARD

Live at aakashmanandhar.tech/aurora

Dashboard

LIVE

React 19 · TypeScript · Vite · Tailwind CSS 4

- A draggable globe, coloured by how unusual each city's last 30 days were
- AI forecast: the measured line runs into the model's range, beside a seven-day list
- Since 1940: warming stripes above yearly bars and a ten-year average
- Can you trust the forecast? The model against simple guesses, day by day

Interaction and resilience

LIVE

Hand-drawn SVG charts · TanStack Query · d3-geo

- Point at any chart, or drag a finger across it, to read exact values
- Linked views highlight the same day or year across charts
- Designed screens for loading, an outage, a city still loading and a forecast not yet made
- An error boundary keeps one failed card from blanking the page

How it's built view

LIVE

A second view, linkable at #about

- Written for recruiters: the data, pipeline, model, serving and DataOps in plain words
- Two live figures from the API: cities loaded and daily records
- Known limits stated plainly, with a link to the code

Hosting

LIVE

Docker Compose · nginx · Cloudflare

- Two containers: nginx serving the built site, and the API, reachable only through it
- The host's nginx routes /aurora to a local-only port; the portfolio site beside it is untouched
- Cloudflare in front. Fonts are bundled with the site, with no third-party requests

7 DATAOPS AND INFRASTRUCTURE: THE FOUNDATION UNDER EVERY STAGE

Everything is code, every change is reviewed, production deploys itself

Continuous integration

DONE

GitHub Actions · Ruff · TypeScript

- Every pull request is linted with Ruff, compiled, validated against both environments and built as a front end
- Nothing merges on a red check
- Ten reviewed pull requests merged so far

Deployment: lakehouse

DONE

Databricks Asset Bundles

- Merging to main deploys jobs and pipelines to production
- One bundle, two targets: catalog, city list and upload address are per-target variables
- Deployments queue and never overlap

Deployment: web server

DONE

cron · shell · Docker Compose

- The server checks the repository every five minutes
- It rebuilds its containers only when API or site code changed
- Pull-based, so no server login is stored in GitHub
- A failed build leaves the old version running and is retried

Infrastructure as code

DONE

Terraform

- Schemas and volumes are defined in Terraform
- One Terraform workspace per environment
- Catalogs are created by hand, because Free Edition's default storage blocks creating them through the API

Two environments

DONE

Unity Catalog · one catalog each

- `aurora_dev`: 7 cities, schedules paused, no upload address
- `aurora`: 40 cities, scheduled daily at 05:30 UTC
- Same code in both. Each has the schemas bronze, silver, gold, ml and serving

Monitoring and alerts

DONE

Databricks email · scheduled GitHub workflow

- Databricks emails when any stage of the daily job fails
- A separate workflow reads the live site at 08:00 UTC and fails if the data is over 30 hours old or the reply is not real API output
- Two independent alarms: each catches what the other cannot

Secrets

DONE

Secret scope · GitHub secrets · server .env

- The upload token lives in a Databricks secret scope and in a root-only file on the server
- The Databricks deployment token lives in GitHub secrets
- No credential is in the repository

Constraints, worked around

DONE

Databricks Free Edition · free API tier

- MLflow model artifacts are routed through the Databricks SDK
- The SQL warehouse and the pipeline run in sequence, never at once
- The backfill is paced at four cities a day to stay inside the API's free allowance

Real issue found and fixed. A rule in the Python .gitignore template silently excluded the front end's lib folder. Everything worked on the laptop; the clean build in CI failed and caught it before go-live.

WHAT IS NEXT

- NOAA station data as a second source
- A fresher feed to close the five-day archive lag
- Retraining on all 40 cities, behind a quality gate
- Encrypting the Cloudflare-to-server link
- Exempting the API from Cloudflare's error page
- Disaster alerts, once the source's licence is confirmed

PLATFORM

Databricks Free Edition · Unity Catalog · Delta Lake · Serverless compute

PIPELINES

Lakeflow Declarative Pipelines · Auto Loader · dbt · SQL · PySpark · Python

MACHINE LEARNING

LightGBM · MLflow · pandas · Quantile regression

SERVING

FastAPI · DuckDB · Parquet · Docker · nginx · Cloudflare

FRONT END

React 19 · TypeScript · Vite · Tailwind CSS 4 · TanStack Query · d3-geo

DATAOPS

GitHub Actions · Asset Bundles · Terraform · Ruff · Git · cron

Designed and developed by **Aakash Manandhar**

aakashmanandhar.tech/aurora · github.com/aakashmanandhar/aurora-platform · linkedin.com/in/aakashmanandhar

Weather data by Open-Meteo.com, CC BY 4.0